Knowledge Library

Knowledge Library

Purpose: provide **approved reference material** that improves quality and consistency **without becoming a code-donor**.

What it is (in AXION terms)

- A curated set of **reference units** ("knowledge items") that agents can consult.
- Each item has metadata that controls:
  - **when it's applicable**
  - **how it may be used** (pattern-only vs reusable)
  - **what must be cited/logged**
  - **whether it is safe for external agents**

What the Knowledge Library must include

1) Knowledge Item Types (what content goes in)

These are the categories that actually help template creation:

A) Patterns and playbooks (pattern-only)

- Architecture patterns (auth, payments, realtime, file uploads)
- Error handling patterns
- Logging/observability patterns
- API design patterns
- Data modeling patterns

Use rule: guidance only; do not copy code blocks unless allowlisted.

B) Examples (restricted by default)

- example template fills
- example specs
- example outputs (render envelopes, selection results)
- example gate reports

Use rule: may be used for structure; copying is blocked unless explicitly allowlisted.

C) Checklists

- security checklist per domain
- compliance checklist (PII, secrets, retention)
- UI/UX checklists (a11y, navigation constraints)
- testing checklists

Use rule: can be freely reused because it's "rules," not expressive content.

D) Definitions / glossary extensions

- terms, aliases, naming conventions
- "what does X mean in AXION"

Use rule: canonical definitions; must match Terms Registry.

E) Test vectors / known pitfalls

- "common failure modes" and how to avoid them
- "gotchas" for tools/frameworks

Use rule: safe guidance.

---

2) Knowledge Index (the registry for the library)

You need a **KNOWLEDGE_INDEX** parallel to your other registries.

Each knowledge item should have:

- `kid` (knowledge id)
- `title`
- `type` (pattern/checklist/example/definition/pitfall)
- `scope_tags` (auth, payments, realtime, mobile, etc.)
- `applicability` (mode/profile/executor type)
- `use_policy`
    - `pattern_only | reusable_with_allowlist | forbidden_for_external`
- `source` (internal authored vs external licensed)
- `license` (if external)
- `allowed_excerpt_limit` (text/code)
- `required_attribution` (if any)
- `redaction_class` (public/internal/restricted)

This is what prevents the library from becoming “random docs.”

---

3) How agents are allowed to use it (core behavior rules)

This is the “study guide” behavior you want.

Allowed

- Learn patterns and constraints
- Translate knowledge into *original* outputs
- Reference checklists and definitions
- Use examples to understand **structure**, not to copy content

Disallowed by default

- Copy code blocks into templates or generated repos
- Copy long prose verbatim into outputs
- Use restricted items when executor is external

Required behavior

- When an agent uses a knowledge item, it must write:
    - `knowledge_citations` into its output envelope OR into a run log
    - (if reuse occurs) `reuse_log` entry

---

4) Knowledge selection (how items get “called”)

You don’t want the agent rummaging.

You want a deterministic selection step:

- input: canonical spec + standards snapshot + run profile + executor type
- output: “recommended knowledge set” (KIDs)

This can be:

- a stage (KSL-SELECT)
- or part of template selection logic

Either way, the selection output should be saved as an artifact:

- `knowledge_selection.json`

---

5) Gates for Knowledge Library safety

Since this library can cause IP issues, it needs enforcement:

- **GATE-KNOW-01**: knowledge items used must be referenced (citations exist)
- **GATE-KNOW-02**: external agents cannot access restricted knowledge items
- **GATE-KNOW-03**: no verbatim copying beyond allowed excerpt limits
- **GATE-KNOW-04**: reuse requires allowlist + reuse log entry

Knowledge Breakdown

Knowledge Library

Purpose: give agents a curated, governed reference system ("study guide") to improve template creation quality **without** turning into a copy-donor.

Core design rules

1. **Atomic items** (small units; no monolith docs)
2. **Indexed and tagged** (selection is deterministic)
3. **Use-policy governed** (pattern-only by default)
4. **Versioned** (reproducibility)
5. **Internal vs external access controlled**
6. **Citations are mandatory when used**

KL-1) Knowledge Item Model

A Knowledge Library entry is a **Knowledge Item** (KID).

KL-1.1 Knowledge Item types (allowed)

- `concept` (what/why)
- `pattern` (reusable approach; pattern-only)
- `procedure` (how-to steps)
- `checklist` (enforceable list)
- `reference` (tables, defaults, parameter lists)
- `pitfall` (common failure modes)

- `example` (restricted; structure learning only unless allowlisted)
- `glossary_term` (canonical definitions; ties into TERMS_REGISTRY)

KL-1.2 Knowledge Item required sections (content contract)

Every KID file must contain:

- **Summary** (1–3 lines)
- **When to use** (applicability)
- **Do / Don't** (guardrails)
- **Core content** (the knowledge)
- **Links** (optional: internal cross-links)
- **Proof / confidence** (tier + references)

KL-1.3 Knowledge Item metadata (enforcement contract)

Each item must include machine-readable metadata (frontmatter or JSON header):

- `kid`
- `title`
- `type`
- `pillar` (IT_END_TO_END | INDUSTRY_PLAYBOOKS | LANGUAGES_AND_LIBRARIES)
- `domains` / `subdomains`
- `tags`
- `maturity` (draft | reviewed | verified | golden)

- `use_policy` (pattern_only | reusable_with_allowlist | restricted_internal_only)
- `executor_access` (internal_only | internal_and_external)
- `license` (internal_owned | MIT | Apache-2.0 | etc.)
- `allowed_excerpt` (max words/lines if any)
- `supersedes` / `deprecated_by`
- `created_at`, `updated_at`, `owner`

KL-2) Knowledge Registries (Source of Truth)

These sit in `knowledge/INDEX/` per your file tree.

KL-2.1 KNOWLEDGE_INDEX (required)

Maps KID → metadata → file path.

- must be the authoritative lookup for:
    - what exists
    - where it lives
    - allowed use policy
    - maturity
    - access constraints

KL-2.2 TAXONOMY (required)

Canonical taxonomy for:

- pillars
- domains/subdomains
- industries
- stacks/framework families
- tag vocabulary and aliases

KL-2.3 BUNDLES_INDEX (optional but recommended)

Curated sets of KIDs for common contexts:

- by run profile (API/Web/Mobile/etc.)
- by risk class (prototype/production/compliance)
- by executor (internal/external)

KL-2.4 SOURCES_INDEX (required if you ingest external content)

Tracks provenance/licensing:

- source type (internal authored, public domain, OSS docs)
- license
- attribution requirements
- allowed reuse level

KL-2.5 DEPRECATIONS + CHANGELOG (required)

- deprecated KIDs → replacements
- library-level change log

KL-3) Knowledge Selection (How agents “use it” safely)

Agents should not roam. The system selects.

KL-3.1 Inputs to selection

- canonical spec signals (domain/feature requirements)
- standards snapshot
- run profile (API/Web/etc.)
- risk class
- executor type (internal vs external)
- target stack (if known)

KL-3.2 Output: KNOWLEDGE_SELECTION artifact (required)

A deterministic artifact like:

- selected KIDs (ordered)
- why each KID was selected (tags/applicability match)
- restrictions attached (use_policy, excerpt limits)
- bundle IDs used (if any)

KL-3.3 Selection rules (minimum)

- filter by executor access
- filter by applicability
- filter by maturity threshold (e.g., production requires reviewed+)

- rank by relevance (tags + domain match)
- cap selection set (avoid overload)

KL-4) Usage Rules (what agents may do)

This is where your plagiarism/IP policy hooks in.

KL-4.1 Default rule

- Knowledge items are **pattern-only** unless explicitly allowlisted.

KL-4.2 Allowed behaviors

- derive requirements and constraints
- follow procedures and checklists
- re-express concepts in original language
- use examples to understand format (not copy)

KL-4.3 Restricted behaviors

- copying code blocks or long prose unless `reusable_with_allowlist`
- using internal-only items with external agents
- including restricted content inside kits for external execution

KL-4.4 Mandatory citation behavior

When a KID is used, agent must emit:

- `knowledge_citations: [KID-...]` in render envelope / run log

- if reuse: update reuse_log

KL-5) Enforcement Gates (Knowledge safety and integrity)

These make the library “real” and safe.

KL-5.1 Integrity gates

- **KL-GATE-01**: every referenced KID exists in KNOWLEDGE_INDEX
- **KL-GATE-02**: all KID metadata valid (type, tags, policies)
- **KL-GATE-03**: no deprecated KIDs used unless explicitly allowed

KL-5.2 Access gates

- **KL-GATE-04**: external executor cannot access restricted KIDs
- **KL-GATE-05**: kit export for external use excludes restricted content

KL-5.3 Plagiarism/reuse gates

- **KL-GATE-06**: if output contains allowlisted excerpts, reuse_log must include entry
- **KL-GATE-07**: block verbatim copying beyond excerpt limits

KL-5.4 Quality gates (optional but strong)

- **KL-GATE-08**: production runs require maturity >= reviewed for selected KIDs

---

KL-6) Scaling Over Time (content ops)

This is how it grows without rotting.

KL-6.1 Growth model

- add items atomically
- update taxonomy conservatively
- use bundles to create curated “slices”

KL-6.2 Maturity workflow

- draft → reviewed → verified → golden
- reviewed means: checked for correctness + policy alignment
- verified means: validated against real runs or tests
- golden means: stable canonical reference; changes are controlled

KL-6.3 Deprecation workflow

- mark deprecated
- add replacement KID
- keep aliases for a window
- block new selection of deprecated KIDs (except repro)

KL-6.4 External content ingestion rules

If you ingest outside material:

- store source metadata
- do not store copyrighted text verbatim unless licensed

- summarize + cite; keep excerpts within allowed limits
- enforce through `SOURCES_INDEX` + use_policy gates

KL-7) Minimum Viable Knowledge Library (what you should build first)

Given your “everything IT + industries + languages” goal, don’t start with “everything.” Start with a scalable spine:

1. **Taxonomy + index system**
2. **IT_END_TO_END**: security, networking, databases, auth, CI/CD, observability (core)
3. **Languages**: TypeScript/Node, React, SQL/Postgres (core)
4. **Industry playbooks**: 3–5 industries to start (healthcare, finance, ecommerce, logistics, gov)

Then scale by adding atomic KIDs and bundles.

KL-1) Knowledge Item Model

KL-1) Knowledge Item Model

A Knowledge Library entry is a **Knowledge Item** (KID).

KL-1.1 Knowledge Item types (allowed)

- `concept` (what/why)
- `pattern` (reusable approach; pattern-only)
- `procedure` (how-to steps)
- `checklist` (enforceable list)
- `reference` (tables, defaults, parameter lists)
- `pitfall` (common failure modes)
- `example` (restricted; structure learning only unless allowlisted)
- `glossary_term` (canonical definitions; ties into TERMS_REGISTRY)

KL-1.2 Knowledge Item required sections (content contract)

Every KID file must contain:

- **Summary** (1–3 lines)
- **When to use** (applicability)
- **Do / Don't** (guardrails)
- **Core content** (the knowledge)
- **Links** (optional: internal cross-links)
- **Proof / confidence** (tier + references)

KL-1.3 Knowledge Item metadata (enforcement contract)

Each item must include machine-readable metadata (frontmatter or JSON header):

- `kid`
- `title`
- `type`
- `pillar` (IT_END_TO_END | INDUSTRY_PLAYBOOKS | LANGUAGES_AND_LIBRARIES)
- `domains` / `subdomains`
- `tags`
- `maturity` (draft | reviewed | verified | golden)
- `use_policy` (pattern_only | reusable_with_allowlist | restricted_internal_only)
- `executor_access` (internal_only | internal_and_external)
- `license` (internal_owned | MIT | Apache-2.0 | etc.)
- `allowed_excerpt` (max words/lines if any)
- `supersedes` / `deprecated_by`
- `created_at`, `updated_at`, `owner`

KL-1.1 Knowledge Item types

KL-1.2 Knowledge Item required sections

KL-1.3 Knowledge Item metadata

KL-2) Knowledge Registries

KL-2) Knowledge Registries (Source of Truth)

These sit in `knowledge/INDEX/` per your file tree.

KL-2.1 KNOWLEDGE_INDEX (required)

Maps KID → metadata → file path.

- must be the authoritative lookup for:
    - what exists
    - where it lives
    - allowed use policy
    - maturity
    - access constraints

KL-2.2 TAXONOMY (required)

Canonical taxonomy for:

- pillars
- domains/subdomains
- industries
- stacks/framework families
- tag vocabulary and aliases

KL-2.3 BUNDLES_INDEX (optional but recommended)

Curated sets of KIDs for common contexts:

- by run profile (API/Web/Mobile/etc.)
- by risk class (prototype/production/compliance)
- by executor (internal/external)

KL-2.4 SOURCES_INDEX (required if you ingest external content)

Tracks provenance/licensing:

- source type (internal authored, public domain, OSS docs)
- license
- attribution requirements
- allowed reuse level

KL-2.5 DEPRECATIONS + CHANGELOG (required)

- deprecated KIDs → replacements
- library-level change log

KL-2.1 KNOWLEDGE_INDEX

KL-2.2 TAXONOMY

KL-2.3 BUNDLES_INDEX

KL-2.4 SOURCES_INDEX

KL-2.5 DEPRECATIONS + CHANGELOG

KL-3) Knowledge Selection

KL-3) Knowledge Selection (How agents “use it” safely)

Agents should not roam. The system selects.

KL-3.1 Inputs to selection

- canonical spec signals (domain/feature requirements)
- standards snapshot
- run profile (API/Web/etc.)
- risk class
- executor type (internal vs external)
- target stack (if known)

KL-3.2 Output: KNOWLEDGE_SELECTION artifact (required)

A deterministic artifact like:

- selected KIDs (ordered)
- why each KID was selected (tags/applicability match)
- restrictions attached (use_policy, excerpt limits)
- bundle IDs used (if any)

KL-3.3 Selection rules (minimum)

- filter by executor access
- filter by applicability
- filter by maturity threshold (e.g., production requires reviewed+)

- rank by relevance (tags + domain match)
- cap selection set (avoid overload)

Tab 18

Tab 19

Tab 20

KL-4) Usage Rules

KL-4) Usage Rules (what agents may do)

This is where your plagiarism/IP policy hooks in.

KL-4.1 Default rule

- Knowledge items are **pattern-only** unless explicitly allowlisted.

KL-4.2 Allowed behaviors

- derive requirements and constraints
- follow procedures and checklists
- re-express concepts in original language
- use examples to understand format (not copy)

KL-4.3 Restricted behaviors

- copying code blocks or long prose unless `reusable_with_allowlist`
- using internal-only items with external agents
- including restricted content inside kits for external execution

KL-4.4 Mandatory citation behavior

When a KID is used, agent must emit:

- `knowledge_citations: [KID-...]` in render envelope / run log
- if reuse: update reuse_log

KL-4.1 Default rule

KL-4.2 Allowed behaviors

KL-4.3 Restricted behaviors

KL-4.4 Mandatory citation behavior

KL-5) Enforcement Gates

KL-5) Enforcement Gates (Knowledge safety and integrity)

These make the library “real” and safe.

KL-5.1 Integrity gates

- **KL-GATE-01**: every referenced KID exists in KNOWLEDGE_INDEX
- **KL-GATE-02**: all KID metadata valid (type, tags, policies)
- **KL-GATE-03**: no deprecated KIDs used unless explicitly allowed

KL-5.2 Access gates

- **KL-GATE-04**: external executor cannot access restricted KIDs
- **KL-GATE-05**: kit export for external use excludes restricted content

KL-5.3 Plagiarism/reuse gates

- **KL-GATE-06**: if output contains allowlisted excerpts, reuse_log must include entry
- **KL-GATE-07**: block verbatim copying beyond excerpt limits

KL-5.4 Quality gates (optional but strong)

- **KL-GATE-08**: production runs require maturity >= reviewed for selected KIDs

Tab 25

Tab 26

Tab 27

Tab 28

KL-6) Scaling Over Time

KL-6) Scaling Over Time (content ops)

This is how it grows without rotting.

KL-6.1 Growth model

- add items atomically
- update taxonomy conservatively
- use bundles to create curated “slices”

KL-6.2 Maturity workflow

- draft → reviewed → verified → golden
- reviewed means: checked for correctness + policy alignment
- verified means: validated against real runs or tests
- golden means: stable canonical reference; changes are controlled

KL-6.3 Deprecation workflow

- mark deprecated
- add replacement KID
- keep aliases for a window
- block new selection of deprecated KIDs (except repro)

KL-6.4 External content ingestion rules

If you ingest outside material:

- store source metadata

- do not store copyrighted text verbatim unless licensed
- summarize + cite; keep excerpts within allowed limits
- enforce through `SOURCES_INDEX` + use_policy gates

KL-6.1 Growth model

KL-6.2 Maturity workflow

KL-6.3 Deprecation workflow

KL-6.4 External content ingestion rules

KL-7) Minimum Viable Knowledge Library

KL-7) Minimum Viable Knowledge Library (what you should build first)

Given your “everything IT + industries + languages” goal, don’t start with “everything.” Start with a scalable spine:

1. **Taxonomy + index system**
2. **IT_END_TO_END**: security, networking, databases, auth, CI/CD, observability (core)
3. **Languages**: TypeScript/Node, React, SQL/Postgres (core)
4. **Industry playbooks**: 3–5 industries to start (healthcare, finance, ecommerce, logistics, gov)

IT_END_TO_END

IT_END_TO_END pillar spec

A) Pillar folder (root)

```
knowledge/PILLARS/IT_END_TO_END/
 _meta/
   pillar.md
   domain_map.json
   selection_rules.md
   bundles.md
```

_meta purpose

- `pillar.md` = what this pillar covers + how it should be used
- `domain_map.json` = authoritative taxonomy (domains/subdomains)
- `selection_rules.md` = how selection should prioritize KIDs (risk class, maturity)
- `bundles.md` = recommended bundles (baseline sets)

B) IT_END_TO_END domain taxonomy (what it covers)

This is the “end-to-end IT” map you want. Keep it stable; expand via subdomains over time.

01 Foundations

1. **Networking**
    - DNS, DHCP, routing, switching, NAT, firewalls, VPN, load balancing
2. **Operating Systems**
    - Linux, Windows, processes, permissions, filesystems
3. **Security fundamentals**
    - IAM, authN/authZ, secrets, encryption, threat models

4. **Compute & virtualization**
    - VMs, containers, virtualization, scheduling basics
5. **Storage fundamentals**
    - block/object/file, durability, backups, recovery basics

02 Software Delivery

6. **Architecture & design**
    - monolith vs microservices, layering, DDD, boundaries
7. **APIs & integration**
    - REST, RPC, webhooks, idempotency, versioning
8. **CI/CD & DevOps**
    - pipelines, deployments, environment parity, rollbacks
9. **Testing & QA**
    - unit/integration/e2e, test strategy, test data
10. **Observability & SRE**

- logs, metrics, traces, SLOs/SLIs, incident response

03 Data Systems

11. **Databases**

- relational, NoSQL, indexing, transactions, migrations

12. **Distributed systems**

- consensus basics, replication, CAP tradeoffs, queues

13. **Caching**

- cache patterns, invalidation, Redis basics, CDN caching

14. **Search & retrieval**

- indexing, ranking basics, search architecture

04 Platform & Operations

15. **Cloud fundamentals**

- AWS/GCP/Azure concepts, shared responsibility, networking in cloud

16. **Identity & access management**

- RBAC, ABAC, SSO/OAuth/OIDC, audit trails

17. **Compliance & governance**

- PII, retention, audit, policy enforcement

18. **Release management**

- versioning, deprecation, migrations, change control

19. **Cost / FinOps**

- cost drivers, capacity planning, usage monitoring

C) Required subfolders (consistent across domains)

Every domain folder uses the same internal layout so agents always know where to look:

```
<domain>/
 _meta.md
 concepts/
 patterns/
 procedures/
 checklists/
 pitfalls/
 references/
```

```
examples/        # restricted by default (pattern learning only)
```

What each folder means

- **concepts**: definitions, mental models, “how it works”
- **patterns**: reusable approaches (pattern-only)
- **procedures**: step-by-step operational tasks
- **checklists**: enforceable lists, “don’t forget X”
- **pitfalls**: failure modes, what breaks and why
- **references**: tables, defaults, comparisons
- **examples**: sample outputs (restricted unless allowlisted)

D) Naming + ID rules for IT_END_TO_END KIDs

Folder-based naming (human-friendly)

File names should start with a KID and be stable:

- `KID-ITNET-CONCEPT-0001.md`
- `KID-ITSEC-CHECK-0012.md`

KID format (machine-friendly)

`KID-<PILLAR><DOMAIN>-<TYPE>-<NNNN>.md`

Examples:

- `KID-ITNET-PATTERN-0007.md` (IT Networking pattern)
- `KID-ITDB-PROCEDURE-0011.md` (IT Databases procedure)

- `KID-ITSRE-PITFALL-0003.md` (SRE pitfall)

E) Domain folders to create (v1 file tree)

This is a clean starting tree that matches the taxonomy:

```
knowledge/PILLARS/IT_END_TO_END/
 _meta/
 01_foundations/
   networking/
   operating_systems/
   security_fundamentals/
   compute_virtualization/
   storage_fundamentals/
 02_software_delivery/
   architecture_design/
   apis_integrations/
   ci_cd_devops/
   testing_qa/
   observability_sre/
 03_data_systems/
   databases/
   distributed_systems/
   caching/
   search_retrieval/
 04_platform_ops/
   cloud_fundamentals/
   identity_access_management/
   compliance_governance/
   release_management/
   finops_cost/
```

Each of those folders contains the standardized subfolders from section C.

F) Minimal bundles for IT_END_TO_END (what agents should start with)

These are curated sets of KIDs you'll build over time:

- **IT-BUNDLE-BASELINE**: networking + security fundamentals + databases + APIs + CI/CD + observability (core for most builds)

- **IT-BUNDLE-PRODUCTION**: baseline + IAM + release mgmt + governance + incident/SRE checklists

- **IT-BUNDLE-COMPLIANCE**: production + compliance/PII/retention/audit heavy KIDs

Pillar folder (root)

Pillar folder (root)

```
knowledge/PILLARS/IT_END_TO_END/
 _meta/
   pillar.md
   domain_map.json
   selection_rules.md
   bundles.md
```

_meta purpose

- `pillar.md` = what this pillar covers + how it should be used
- `domain_map.json` = authoritative taxonomy (domains/subdomains)
- `selection_rules.md` = how selection should prioritize KIDs (risk class, maturity)
- `bundles.md` = recommended bundles (baseline sets)

IT_END_TO_END domain taxonomy

IT_END_TO_END domain taxonomy (what it covers)

This is the “end-to-end IT” map you want. Keep it stable; expand via subdomains over time.

01 Foundations

1. **Networking**
    - DNS, DHCP, routing, switching, NAT, firewalls, VPN, load balancing
2. **Operating Systems**
    - Linux, Windows, processes, permissions, filesystems
3. **Security fundamentals**
    - IAM, authN/authZ, secrets, encryption, threat models
4. **Compute & virtualization**
    - VMs, containers, virtualization, scheduling basics
5. **Storage fundamentals**
    - block/object/file, durability, backups, recovery basics

02 Software Delivery

6. **Architecture & design**
    - monolith vs microservices, layering, DDD, boundaries
7. **APIs & integration**
    - REST, RPC, webhooks, idempotency, versioning
8. **CI/CD & DevOps**
    - pipelines, deployments, environment parity, rollbacks
9. **Testing & QA**

    - unit/integration/e2e, test strategy, test data

10. **Observability & SRE**

- logs, metrics, traces, SLOs/SLIs, incident response

03 Data Systems

11. **Databases**

- relational, NoSQL, indexing, transactions, migrations

12. **Distributed systems**

- consensus basics, replication, CAP tradeoffs, queues

13. **Caching**

- cache patterns, invalidation, Redis basics, CDN caching

14. **Search & retrieval**

- indexing, ranking basics, search architecture

04 Platform & Operations

15. **Cloud fundamentals**

- AWS/GCP/Azure concepts, shared responsibility, networking in cloud

16. **Identity & access management**

- RBAC, ABAC, SSO/OAuth/OIDC, audit trails

17. **Compliance & governance**

- PII, retention, audit, policy enforcement

18. **Release management**

- versioning, deprecation, migrations, change control

19. **Cost / FinOps**

- cost drivers, capacity planning, usage monitoring

Required subfolders

Required subfolders (consistent across domains)

Every domain folder uses the same internal layout so agents always know where to look:

```
<domain>/
 _meta.md
 concepts/
 patterns/
 procedures/
 checklists/
 pitfalls/
 references/
 examples/        # restricted by default (pattern learning only)
```

What each folder means

- **concepts**: definitions, mental models, “how it works”
- **patterns**: reusable approaches (pattern-only)
- **procedures**: step-by-step operational tasks
- **checklists**: enforceable lists, “don’t forget X”
- **pitfalls**: failure modes, what breaks and why
- **references**: tables, defaults, comparisons
- **examples**: sample outputs (restricted unless allowlisted)

Naming + ID rules for IT_END_TO_END KIDs

Naming + ID rules for IT_END_TO_END KIDs

Folder-based naming (human-friendly)

File names should start with a KID and be stable:

- `KID-ITNET-CONCEPT-0001.md`
- `KID-ITSEC-CHECK-0012.md`

KID format (machine-friendly)

`KID-<PILLAR><DOMAIN>-<TYPE>-<NNNN>.md`

Examples:

- `KID-ITNET-PATTERN-0007.md` (IT Networking pattern)
- `KID-ITDB-PROCEDURE-0011.md` (IT Databases procedure)
- `KID-ITSRE-PITFALL-0003.md` (SRE pitfall)

Domain folders to create

Domain folders to create (v1 file tree)

This is a clean starting tree that matches the taxonomy:

```
knowledge/PILLARS/IT_END_TO_END/
 _meta/
 01_foundations/
   networking/
   operating_systems/
   security_fundamentals/
   compute_virtualization/
   storage_fundamentals/
 02_software_delivery/
   architecture_design/
   apis_integrations/
   ci_cd_devops/
   testing_qa/
   observability_sre/
 03_data_systems/
   databases/
   distributed_systems/
   caching/
   search_retrieval/
 04_platform_ops/
   cloud_fundamentals/
   identity_access_management/
   compliance_governance/
   release_management/
   finops_cost/
```

Each of those folders contains the standardized subfolders from section C.

Minimal bundles for IT_END_TO_END

Minimal bundles for IT_END_TO_END (what agents should start with)

These are curated sets of KIDs you'll build over time:

- **IT-BUNDLE-BASELINE**: networking + security fundamentals + databases + APIs + CI/CD + observability (core for most builds)
- **IT-BUNDLE-PRODUCTION**: baseline + IAM + release mgmt + governance + incident/SRE checklists
- **IT-BUNDLE-COMPLIANCE**: production + compliance/PII/retention/audit heavy KIDs

INDUSTRY_PLAYBOOKS

LANGUAGES_AND_LIBRARIES

Security KIDs

Security Fundamentals — KID Starter Set (v1)

concepts/

- **KID-ITSEC-CONCEPT-0001 — Threat Modeling Basics (assets, actors, attack surfaces)**
- **KID-ITSEC-CONCEPT-0002 — AuthN vs AuthZ (what they are, common failure patterns)**
- **KID-ITSEC-CONCEPT-0003 — Principle of Least Privilege (POLP)**
- **KID-ITSEC-CONCEPT-0004 — Defense in Depth**
- **KID-ITSEC-CONCEPT-0005 — CIA Triad + Practical Mapping to Systems**
- **KID-ITSEC-CONCEPT-0006 — Secrets vs Credentials vs Tokens (taxonomy)**
- **KID-ITSEC-CONCEPT-0007 — Encryption Basics (at rest, in transit, key roles)**

patterns/

- **KID-ITSEC-PATTERN-0001 — Security Baseline by Default (secure defaults checklist)**
- **KID-ITSEC-PATTERN-0002 — Auth Session Hardening Pattern (rotation, expiry, refresh)**
- **KID-ITSEC-PATTERN-0003 — RBAC Pattern (roles, permissions, least privilege)**
- **KID-ITSEC-PATTERN-0004 — ABAC Pattern (attributes + policy evaluation)**
- **KID-ITSEC-PATTERN-0005 — API Key Management Pattern (issue, rotate, revoke)**
- **KID-ITSEC-PATTERN-0006 — Rate Limit + Abuse Control Pattern (token bucket + bans)**
- **KID-ITSEC-PATTERN-0007 — Audit Logging Pattern (what to log, what not to log)**
- **KID-ITSEC-PATTERN-0008 — Input Validation + Output Encoding Pattern**

- **KID-ITSEC-PATTERN-0009 — Webhook Verification Pattern (signatures, replay protection)**

procedures/

- **KID-ITSEC-PROCEDURE-0001 — Secrets Handling Procedure (dev, CI, prod)**
- **KID-ITSEC-PROCEDURE-0002 — Incident Triage Procedure (containment → recovery)**
- **KID-ITSEC-PROCEDURE-0003 — Key Rotation Procedure (what triggers rotation)**
- **KID-ITSEC-PROCEDURE-0004 — Vulnerability Response Procedure (patch cycle + verification)**
- **KID-ITSEC-PROCEDURE-0005 — Access Review Procedure (who has what, why)**
- **KID-ITSEC-PROCEDURE-0006 — Secure Release Checklist Procedure (pre-release checks)**

checklists/

- **KID-ITSEC-CHECK-0001 — Auth System Checklist (sessions, tokens, MFA readiness)**
- **KID-ITSEC-CHECK-0002 — API Security Checklist (auth, rate limit, input validation, logging)**
- **KID-ITSEC-CHECK-0003 — Data Handling Checklist (PII, retention, redaction)**
- **KID-ITSEC-CHECK-0004 — Logging Checklist (no secrets, correlation IDs, audit fields)**
- **KID-ITSEC-CHECK-0005 — Dependency Hygiene Checklist (lockfiles, updates, license notes)**
- **KID-ITSEC-CHECK-0006 — Threat Model Checklist (assets, actors, mitigations)**
- **KID-ITSEC-CHECK-0007 — External Agent Execution Checklist (least privilege + no copy rules)**

pitfalls/

- **KID-ITSEC-PITFALL-0001 — “JWT Everywhere” Failure Modes**
- **KID-ITSEC-PITFALL-0002 — Secrets in Logs (how it happens, prevention)**
- **KID-ITSEC-PITFALL-0003 — Broken Access Control (most common causes)**
- **KID-ITSEC-PITFALL-0004 — Missing Idempotency on Webhooks**
- **KID-ITSEC-PITFALL-0005 — Over-permissive CORS and Public RPC**
- **KID-ITSEC-PITFALL-0006 — Insecure Defaults in Dev That Ship to Prod**

references/

- **KID-ITSEC-REF-0001 — Security Terminology Reference (canonical definitions + aliases)**
- **KID-ITSEC-REF-0002 — Common HTTP Security Headers Reference**
- **KID-ITSEC-REF-0003 — Password Storage Reference (hashing requirements, anti-patterns)**
- **KID-ITSEC-REF-0004 — OWASP Top Risks Reference (mapped to common app modules)**

examples/ (restricted by default)

- **KID-ITSEC-EXAMPLE-0001 — Example Threat Model (small web app)**
- **KID-ITSEC-EXAMPLE-0002 — Example Audit Log Event Schema**
- **KID-ITSEC-EXAMPLE-0003 — Example RBAC Permission Matrix**

Tab 43